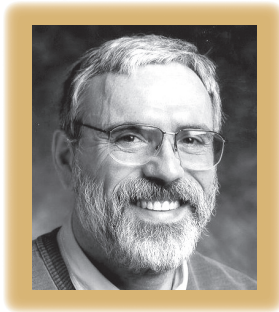


Agile Documentation, Anyone?

Bran Selic

We all warmed to the speaker as he listed the many hurdles facing today's software developers. The audience, experienced programmers all, were happy to hear someone speak so frankly about the absurd demands imposed on them by an uncomprehending, pointy-haired management. Emboldened by our support, the speaker continued confidently: "... and," he proclaimed, "developers don't like to do documentation, because it has no value for them." His listeners nodded enthusiastically, exchanging smiles and knowing glances. Yes, 'tis indeed so, you could hear them think.



Wait! I thought, isn't the primary purpose of design documentation to help *others*? It's rarely created for its authors' benefit—although sometimes the process of explaining things to others can help us clarify and organize our thoughts. How is it possible that there's such a distorted view of documentation?

Still, after 30 years as a developer of large industrial software, this self-centered and disdainful attitude toward documentation didn't surprise me. Like finicky domestic helpers who announce that they "don't do windows," I've often heard software developers state proudly that they "don't do documentation," even refusing lucrative job offers that stipulated it. This viewpoint is echoed in the widely praised Agile Manifesto (<http://agilemanifesto.org>), issued by the software industry's avant-garde, which proclaims: "Working software over comprehensive documentation"—as if these two are necessarily mutually exclusive!

Why Document Designs?

In other engineering disciplines, the need to pro-

duce design documentation is considered *de rigeur*, a sometimes unpleasant but necessary chore for the sake of higher good. It's well understood that the purpose of documentation is to instruct those who are unfamiliar with a system (or who may have forgotten it) about how the system is structured, how it works, and the design rationale that led to it. The primary consumers of design documentation are those who are responsible for maintaining and evolving the system. However, other stakeholders, such as users, testers, and project managers, and, yes, even salespeople may also need to know about a system's design.

The only alternative to design documentation is discovery by exploring the system, akin to trying to find one's way through a jungle without a map or compass. This task is difficult even when experts are available to guide and instruct the novice. System knowledge gained in this way is usually unstructured, incomplete, and inconsistent. It's also inefficient, since experts might need to repeat their instructions over and over again for different groups. Thus, while producing design documentation comes at a cost, the investment—if done properly—usually pays off substantially in the future.

Moreover, when it comes to software, the need for good design documentation seems even more pressing, considering that

- modern software is increasing in complexity,
- software systems are typically revisited and revised more often than other types of systems, and
- code maintenance is usually delegated to less experienced junior staff who are unfamiliar with the code.

Yet, the trend in practice seems to be headed in the opposite direction. The rationale given is the difficulty of keeping documentation in sync

with something that is as highly dynamic as software, and the cost of producing documentation when the same time and resources could be used to generate more code (“working software over comprehensive documentation”). Finally, there are those who argue that code is its own documentation, the only one that contains the real truth.

Anyone familiar with industrial software will recognize these arguments as bogus. Industry is replete with stories of decades-old legacy code that no one dares touch because no one understands it. The intellectual effort required to understand anything but the simplest code through direct inspection can be substantial—even in cases of well-documented code (which, sadly, is the exception rather than the rule). The problem, of course, is that the semantics of the core concepts of popular computer languages are usually far removed from the semantics of the application domain. The breadth of this gap is the reason why we still need programmers. Furthermore, there’s the added complexity of coping with the sometimes esoteric and unintuitive syntax of programming languages, which isn’t always driven by its communication value for human readers but by the needs of compilation technology (for example, single-pass compilation). Thus, when inspecting code, the human reader must replicate a compiler’s parsing function, while at the same time trying to understand the code at the higher application level. And, even when we finally discern how some program fragment works, we still might not understand the rationale behind its design.

The preference for code over documentation in the Agile Manifesto is based on the conclusion that it’s more effective to have something concrete that works and which can be objectively assessed and modified than it is to assess and modify something that’s merely a plan. This is indeed so, but it only argues against a specific category of design documentation: those voluminous and detailed documents describing *planned* implementations, created and polished before there’s an understanding of the problem or experience with the solution. Documents of this type quickly become obsolete, and much of the effort invested in producing and updating

them is wasted. As Helmuth von Moltke noted, “No plan survives the first contact with the enemy.” In this case, the “enemy” is the reality exposed by working code, which can quickly uncover issues that are difficult to anticipate in design.

Still, the ineffectiveness of such “blind” a priori design documents isn’t an argument for eliminating design documentation or making it superficial. It’s interesting that the authors of the manifesto used the phrase “comprehensive documentation,” which can be traced to the root “to comprehend”—that is, to understand. Isn’t rapid understanding what all agile processes are striving for? In agile development, we reach understanding through experimentation with working systems—undeniably a most effective way to learn. However, this path is available only to the original designers, not to those who follow in their footsteps or those who are unable to participate directly in the design.

Agile Documentation

So, what type of documentation is useful, what should it look like, and when should it be produced?

Clearly, documenting software at the code level is foolish; this simply results in the nightmare of trying to keep duplicated information consistent (code comments often fall into this useless category). Instead, we need design documentation at a higher level of abstraction, stripped of unnecessary technological detail and closely coupled to application concepts and requirements. These should incorporate design rationale, including descriptions of rejected design alternatives. These are, in fact, *architectural specifications*: technology-independent descriptions of the higher-level structure and behavior of systems along with key design principles.

Such documents not only help us see the forest hiding in a sea of code but, by documenting the core design principles, they help us maintain consistency when making finer-grained design decisions. Furthermore, architectural specifications serve as the primary medium by which the diverse system stakeholders communicate with the design team and with each other. With the exception of relatively small systems that can be prototyped and experimented with quickly, doing high-level design by coding (as recommended, for

example, in Extreme Programming¹) is unwise. The semantic distance that must be covered between working code and something that stakeholders find meaningful and can evaluate is too vast to be covered easily, particularly if short iterations are used to achieve agility.

In effect, I’m talking about *agile design documentation*, which keeps step with design and development. To avoid waste, the effort expended on such documents should be proportional to the design’s stability. That is, the more stable the design, the more extensive the documentation. However, even in the earliest stages, the documentation must be adequate to serve its primary communications purpose.

One promising approach to this is based on using higher-level domain-specific modeling languages that produce executable models. This involves rapid construction of architectural models (sometimes requiring relatively small code fragments embedded and executed along with the model) that can be evaluated in the usual agile manner. This avoids the high overhead of coding complex structures using programming languages. Such models can then be easily integrated into documents using modern customizable document generation systems, along the lines of Donald Knuth’s *literate programming*.²

In our rush to code and produce something that works—and software is exceptional in its ability to short-circuit the path from idea to realization—we must take care not to discard something that the long history of engineering has established as fundamentally useful. Instead, we must adapt it to our medium and processes. 

References

1. K. Beck, *eXtreme Programming Explained*, Addison-Wesley, 2000.
2. D. Knuth, “Literate Programming,” *The Computer J.*, vol. 27, no. 2, pp. 97–111.

Bran Selic is president of Malina Software Corp., which provides consulting services to Fortune 100 clients worldwide, and an adjunct professor of computer science at Carleton University and the University of Toronto. Contact him at selic@ieee.org.